Topic: News Feed System

What is News Feed?

A news feed is a stream of continuously updated content, typically displayed on a webpage or app, showing the latest information from various sources.

4 steps:
Functional requirements(实现什么功能
Non-Function Requirement
Back-of-the-envelope estimation（scale的计算，加减乘除）
    a. 要多少resource
High-Level Design

Functional requirements

Understand the problem and establish design scope (CRUD)

- Users see a personalized feed with recent posts from people they follow.
    - **Read**: Fetching posts from Feed Storage
    - Involves querying, filtering, ranking — but no data is modified by the user
    - This is a **read-heavy** operation
- Posts should be ordered by time and/or relevance
    - Also **Read**, enhanced with sorting and ranking
    - The ordering logic is handled in the **Feed Service + Ranking Engine**
- Support for liking, commenting, and resharing postes
    - **Create**: Add new like, comment, or share
    - **resharing** = **Create a new reference**, not **update the original post**.
    - **Read**: View existing likes/comments

What does the system do?

It delivers a personalized news feed showing users the most relevant and recent posts from the accounts they follow. The feed is ordered by time and/or relevance, and supports engagement features like liking, commenting, and resharing.

Who uses the system?

- **Public Users**: General users who view and interact with the news feed.

How do they use the system?

- Users open the app or website and are served a personalized feed.

- They scroll through the list of posts, and can like, comment on, or reshare posts.

Non-Function Requirement

(always come after fix function requirements)

- Low latency. Users should be able to see nearby businesses quickly.
- High availability and scalability requirements. Should ensure the system can handle the spike in traffic during peak hours in densely populated areas.

Back-of-the-envelope estimation

QPS, TPS (每秒request
日均一亿，每秒5000
每秒大于300，就得需要很多的云服务

- Seconds in a day = 24 * 60 * 60 = 86400, 10 ^ 5 for easier calculation.
- Each view = 1 query to Redis or Feed Storage

Read Load (Feed Views):

- 100 QPS 10 ^ 6 queries/day

Write Load (Post Creation):

- 1M new posts/day = ~12 QPS for post creation

High-Level Design

API Design

- **GET /api/v1/feed?user_id={user_id}**:
  - Retrieve personalized feed for a user
- **POST /api/v1/posts/{post_id}/like**:
  - Like a specific post
- **POST /api/v1/posts/{post_id}/comment**:
  - Add a comment to a post
- **POST /api/v1/posts/{post_id}/reshare**:
  - Reshare a post to user's feed

Contract Design

- Feed response includes: post_id, author_id, content, timestamp, like_count, comment_count
- Example: Request body: GET /api/v1/feed?user_id=123456

- Response Body:
- {

  "user_id": "123456",

  "feed": [

  {

  "post_id": "abc123",

  "author_id": "user789",

  "content": "Just got back from vacation!",

  "timestamp": "2025-08-07T12:34:56Z",

  "like_count": 42,

  "comment_count": 5,

  "reshared_by": null

  }

  ]

  }
- Like/comment/reshare requests include: user_id, post_id, action type

Data Model

Read/Write Ratio

- The system is **read-heavy**:
  - Users frequently open the app to view feeds (high QPS).
  - Reads dominate due to frequent feed views and low write/update rates.
- **Writes are moderate**:
  - Posts are created at a consistent but lower rate.
  - Engagement actions (likes, comments, reshares) are relatively lightweight.

For this read-heavy pattern, using a combination of wide-column storage (e.g. Cassandra) for feed timelines and object storage (e.g. S3) for post metadata is a good architectural fit.

Data Schema

- **Post**: post_id, author_id, content, timestamp
- **FeedEntry**: user_id, post_id, source (who posted it), timestamp
- **Like**: user_id, post_id, timestamp
- **Comment**: comment_id, post_id, user_id, content, timestamp
- **Reshare**: user_id, post_id, original_author_id, timestamp

User Flow

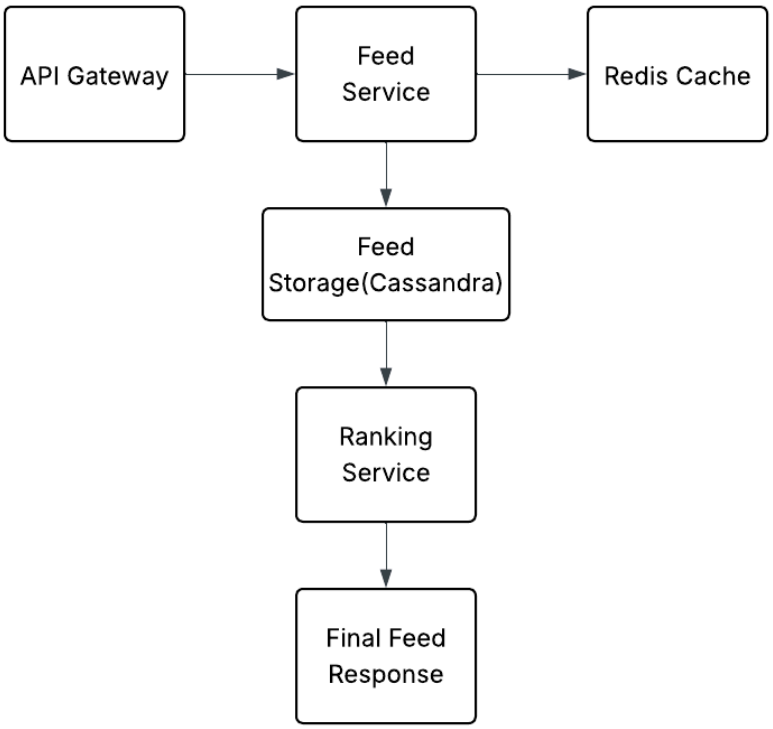
This is a **step-by-step description of how a user interacts with the system**.

1. User opens the app and logs in.
2. The app sends a **GET /feed** request to the backend.
3. The backend checks Redis (cache) for that user's feed.
4. If not found, it queries Feed Storage.
5. The feed is ranked and returned to the user.
6. User scrolls, likes, or comments.

Data Flow

This shows **how data moves through the system** when events occur — like when a post is created.

1. A **`GET /api/v1/feed?user_id={user_id}`** request is received at the API Gateway.
2. The API Gateway forwards the request to the **Feed Service**.
3. The Feed Service checks **Redis**:
    - If cache **hit**, returns feed data immediately.
    - If cache **miss**, queries Feed Storage (Cassandra).
4. The Feed Service retrieves metadata references (e.g. post_id, author_id, timestamp).
5. The Feed Service sends the data to the **Ranking Service** to reorder posts based on recency or relevance.
6. Final feed metadata is returned to the frontend.
7. Full post content may be retrieved asynchronously from **Post Storage** if not cached client-side.


API Gateway
Feed Service
Redis Cache
Feed Storage(Cassandra)
Ranking Service
Final Feed Response